

Two-Person Connection App

V1 Implementation & Learning Guide

A detailed, phase-by-phase engineering guide for building the locked V1 on iOS and Android.

Core product principle: connection without pressure. The app should make affection easier without turning affection into a score, streak, surveillance system, or obligation.

Primary stack: Flutter + Dart · Supabase · PostgreSQL · Realtime · Storage · Edge Functions · Cron/pg_cron

Client architecture: feature-oriented MVVM using Views, ViewModels, Repositories, and Services, with Riverpod for concrete state/dependency management.

Audience: the developer building the app while learning the underlying technologies.

Prepared August 2026

0. Read This First: Locked V1

This guide treats the V1 Product Requirements Document as the source of truth. The implementation should not grow new features while these phases are underway. Finish the core product, prove the two-device experience, then consider expansion.

V1 feature set

- Cross-platform iOS + Android application.
- Exactly two connected partners.
- Account setup and private couple pairing.
- Full-body default avatar + selected pet.
- Partner avatar + pet present across relevant screens with ambient animations.
- Daily outfit colour selection on first opening, reflected on the partner avatar.
- Love Drop: quick affection options + custom text.
- One custom daily question, with question creator alternating by day.
- Answers remain hidden until both partners answer.
- At least one picture from each person's day.
- Mood Sync: mood + optional need + optional context.
- 'I Want to Talk' signal.
- Realtime synchronization.
- Scheduled backend daily jobs.

Explicitly not V1

- No relationship score, love score, reply-time tracking, streaks, or competitive gamification.
- No location tracking, last-seen, or activity surveillance.
- No daily tasks/challenges.
- No mandatory chat system.
- No social/multi-couple product.

How to use the guide

- 1 Implement only the current phase.
- 2 Run the checkpoint on real devices where possible.
- 3 Test both directions using two accounts.
- 4 Test failure and unauthorized paths.
- 5 Commit the work to Git.
- 6 Read the Tech Stack Lesson and explain the design back to yourself before continuing.

Table of Contents

0. Read This First: Locked V1	2
V1 feature set	2
Explicitly not V1	2
How to use the guide	2
Table of Contents	3
Chapter 1 · Phase 0: Project Foundation	6
1.1 Why we start here	6
1.2 Create and verify the project	6
1.3 Recommended structure	6
1.4 The architectural contract	6
1.5 Dependencies	7
1.6 Configuration	7
1.7 Exit checklist	7
Tech Stack Lesson: Flutter + Dart + MVVM + Riverpod	7
Chapter 2 · Phase 1: Supabase Foundation + Authentication	9
2.1 Create the Supabase project	9
2.2 Initialize the client	9
2.3 Authentication flow	9
2.4 User profile table	9
2.5 Repository boundary	9
2.6 RLS	9
2.7 Test cases	10
Tech Stack Lesson: Supabase Auth + PostgreSQL + RLS	10
Chapter 3 · Phase 2: Couple Pairing + Security Boundary	11
3.1 Couple model	11
3.2 Pairing flow	11
3.3 Pairing token	11
3.4 The critical security rule	11
3.5 Exit checklist	12
Tech Stack Lesson: Relational authorization + secure pairing	12
Chapter 4 · Phase 3: Home + Avatar/Pet Presence	13
4.1 Onboarding	13
4.2 Data-driven configuration	13
4.3 Home composition	13
4.4 Animation state machine	13
4.5 Event-driven animation	13
4.6 First milestone	14
Tech Stack Lesson: Flutter rendering + animations + state machines	14

Chapter 5 • Phase 4: Realtime Connection Features	15
5.1 Realtime mental model	15
5.2 Love Drop schema	15
5.3 Love Drop flow	15
5.4 Mood Sync	15
5.5 I Want to Talk	15
5.6 Reconnect	16
Tech Stack Lesson: Supabase Realtime + Streams + event-driven UI	16
Chapter 6 • Phase 5: Daily Question Engine + Cron	17
6.1 Data model	17
6.2 Alternating creator	17
6.3 Cron's job	17
6.4 Idempotency	17
6.5 Answer reveal	17
6.6 State machine	18
6.7 Race cases	18
Tech Stack Lesson: PostgreSQL constraints + Cron + idempotency	18
Chapter 7 • Phase 6: Daily Photo + Outfit Colour Sync	19
7.1 Photo architecture	19
7.2 Never store photo bytes in Postgres	19
7.3 'At least one photo'	19
7.4 Image handling	19
7.5 Outfit colours	19
7.6 Timezone consistency	20
Tech Stack Lesson: Supabase Storage + private media + image processing	20
Chapter 8 • Phase 7: Notifications + UX Polish	21
8.1 Notification matrix	21
8.2 Realtime vs push	21
8.3 Notification privacy	21
8.4 UX states	21
8.5 Cross-platform checks	21
Tech Stack Lesson: Push notifications + mobile lifecycle	22
Chapter 9 • Phase 8: End-to-End Testing + Release	23
9.1 Test with two real accounts	23
9.2 Authentication	23
9.3 Security	23
9.4 Daily question	23
9.5 Photo + avatar	24
9.6 Observability	24
9.7 Release sequence	24

Tech Stack Lesson: Testing, environments, security, operations	24
Chapter 10 · Architecture Deep Dive	26
10.1 End-to-end request	26
10.2 Why repositories?	26
10.3 Where business rules belong	26
10.4 Why not Spring Boot for V1?	26
10.5 Why Flutter rather than native Android + iOS?	26
10.6 Why not build chat?	27
10.7 The storage boundary	27
Chapter 11 · Learning Curriculum	28
The learning method	28
Do not over-learn before building	28
Chapter 12 · Master V1 Checklist	29
Phase 0	29
Phase 1	29
Phase 2	29
Phase 3	29
Phase 4	29
Phase 5	30
Phase 6	30
Phase 7	30
Phase 8	30
Appendix · References & Source Basis	32
Official documentation	32

Chapter 1 - Phase 0: Project Foundation

Goal: establish the cross-platform Flutter codebase and architecture boundaries before feature development.

1.1 Why we start here

The app has several realtime features, media, scheduled jobs, animations, and security-sensitive rules. If we begin with screens and add architecture later, Supabase calls and state will leak into widgets. The result is fast initially and painful afterward.

1.2 Create and verify the project

```
flutter doctor
flutter create two_person_connection
cd two_person_connection
flutter run
git init
git add .
git commit -m "chore: initialize Flutter project"
```

Run Android first. If you have macOS + Xcode, run iOS too. The first milestone is simply: both platforms launch the same codebase.

1.3 Recommended structure

```
lib/
  app/
    app.dart
    router.dart
  core/
    config/
    errors/
    utils/
  features/
    auth/
      data/
      presentation/
    couple/
      data/
      presentation/
    home/
      data/
      presentation/
    avatar/
      data/
      presentation/
    love_drop/
      data/
      presentation/
    mood/
      data/
      presentation/
    daily_question/
      data/
      presentation/
    daily_photo/
      data/
      presentation/
    outfit/
      data/
      presentation/
  shared/
    widgets/
    models/
  main.dart

supabase/
  migrations/
  functions/
```

1.4 The architectural contract

Layer	Owns	Must not own
View	Widgets, layout, simple presentation decisions, animation rendering	Supabase queries, business rules
ViewModel	UI state, commands, presentation logic	Raw database access
Repository	Feature data source and application data contract	Widget rendering
Service	Supabase/platform API calls	Feature UI state
Database	Persistent truth, constraints, authorization	Flutter UI behavior

1.5 Dependencies

```
flutter pub add supabase_flutter
flutter pub add flutter_riverpod
flutter pub add go_router
flutter pub add image_picker
flutter pub add image
```

Add notification and animation-specific packages only when their phases arrive. Avoid dependency accumulation without a reason.

1.6 Configuration

Keep the Supabase URL and publishable key behind build-time configuration. Never put a service-role key in the mobile app. A client key does not make your database public by itself; RLS must still enforce access.

1.7 Exit checklist

- Android build works.
- iOS build works where available.
- Git repository initialized.
- Feature-oriented folders created.
- Riverpod and routing are configured.
- Configuration boundary exists.
- No privileged backend secret is in the app.

Tech Stack Lesson: Flutter + Dart + MVVM + Riverpod

What we are using

- Flutter for one iOS/Android UI codebase.
- Dart for the application language and async programming.
- MVVM-style separation: View, ViewModel, Repository, Service.
- Riverpod for dependency injection and reactive state.
- `go_router` for navigation boundaries.

Why this choice

Flutter is a strong fit because the product has one shared feature set and a highly custom animated UI. The architecture prevents widgets from becoming backend clients. Riverpod gives explicit dependencies and testable state. We are choosing a practical structure, not an architecture ceremony.

What to learn from this phase

- Dart Futures, Streams, async/await.
- Declarative Flutter UI.
- Widget lifecycle.
- MVVM responsibilities.
- Dependency injection and repositories.

Checkpoint: A blank app launches on the target platforms and the codebase has clear UI/data boundaries.

Chapter 2 - Phase 1: Supabase Foundation + Authentication

Goal: establish identity, sessions, the first database table, and database-level access control.

2.1 Create the Supabase project

Create a development Supabase project. Keep production separate. Record the project URL and publishable key from the Connect settings.

2.2 Initialize the client

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Supabase.initialize(
    url: const String.fromEnvironment('SUPABASE_URL'),
    publishableKey:
      const String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY'),
  );

  runApp(const ProviderScope(child: App()));
}
```

2.3 Authentication flow

```
Splash
|
+-- session exists --> onboarding/home decision
|
+-- no session -----> login
                        |
                        v
                    sign up
                        |
                        v
                    profile setup
```

2.4 User profile table

```
create table public.users (
  id uuid primary key references auth.users(id) on delete cascade,
  name text not null,
  avatar_config jsonb not null default '{} '::jsonb,
  pet_config jsonb not null default '{} '::jsonb,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);
```

The profile table is application data; Supabase Auth remains the identity system. Keep those responsibilities separate.

2.5 Repository boundary

```
abstract class AuthRepository {
  Stream<AuthState> get authState;
  Future<void> signIn(String email, String password);
  Future<void> signUp(String email, String password);
  Future<void> signOut();
}
```

The ViewModel calls the repository. The View never calls Supabase directly.

2.6 RLS

```
alter table public.users enable row level security;

create policy "read own profile"
on public.users for select
to authenticated
using (id = auth.uid());
```

```
create policy "update own profile"
on public.users for update
to authenticated
using (id = auth.uid())
with check (id = auth.uid());
```

Enable RLS before client access. Test it rather than assuming it works.

2.7 Test cases

- Valid signup/login.
- Duplicate email.
- Invalid password.
- App restart restores session.
- Sign out removes protected access.
- User can read/update only their own profile.
- Crafted request for another user's profile is denied.
- Network failure is rendered as an error/retry state.

Tech Stack Lesson: Supabase Auth + PostgreSQL + RLS

What we are using

- Supabase Auth for identity/session.
- PostgreSQL for relational application data.
- RLS for database authorization.
- supabase_flutter as the Flutter integration.
- Repositories as the app's data boundary.

Why this choice

For a two-person product, Supabase avoids building and operating a custom authentication service, database API, websocket layer, object store, and job runner. PostgreSQL remains a real relational database, so the project still teaches serious backend fundamentals. RLS is critical because a mobile client is untrusted.

What to learn from this phase

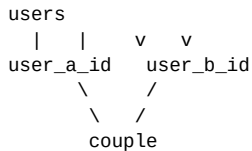
- Sessions and tokens.
- Primary/foreign keys.
- RLS USING vs WITH CHECK.
- Authentication vs authorization.
- Why the client cannot be trusted.

Checkpoint: Two accounts can authenticate, persist sessions, manage their own profiles, and cannot access another user's private profile.

Chapter 3 - Phase 2: Couple Pairing + Security Boundary

Goal: create the private two-person relationship and make couple membership the authorization boundary for every future feature.

3.1 Couple model



```
create table public.couples (
  id uuid primary key default gen_random_uuid(),
  user_a_id uuid not null references public.users(id),
  user_b_id uuid references public.users(id),
  created_at timestamptz not null default now(),
  check (user_b_id is null or user_a_id <> user_b_id)
);
```

3.2 Pairing flow

- 1 User A chooses Create Couple.
- 2 Backend creates a random short-lived pairing token.
- 3 User A shares it privately.
- 4 User B chooses Join Couple and submits it.
- 5 Backend validates expiry, unused status, and eligibility.
- 6 User B becomes the second member.
- 7 The token is marked used.
- 8 Both clients refresh couple state.

3.3 Pairing token

Treat the token as a temporary secret. Prefer storing a hash of the token rather than the raw token, with an expiry and used_at timestamp.

```
pairing_tokens
  id
  couple_id
  token_hash
  created_by
  expires_at
  used_at
```

3.4 The critical security rule

Never treat couple_id as permission. It is only an identifier. Permission comes from proving that auth.uid() is user_a_id or user_b_id for that couple.

```
-- Conceptual authorization predicate
exists (
  select 1
  from couples c
  where c.id = target_couple_id
  and (c.user_a_id = auth.uid()
  or c.user_b_id = auth.uid())
)
```

Every later table should use this same membership idea. Test the policy with a third account.

3.5 Exit checklist

- Create Couple works.
- Join Couple works.
- Token expires.
- Token cannot be replayed.
- Self-pairing is blocked.
- A third user cannot read or mutate the couple.
- A user cannot create multiple active couples if V1 forbids it.

Tech Stack Lesson: Relational authorization + secure pairing

What we are using

- Foreign keys and database constraints.
- RLS policies based on `auth.uid()`.
- Short-lived pairing tokens.
- Least-privilege access.
- Database invariants instead of UI assumptions.

Why this choice

The product is fundamentally a private two-member graph. If couple membership becomes the security boundary, Love Drops, moods, photos, questions, and avatar state can all inherit one coherent authorization model. A short-lived pairing token avoids turning a permanent identifier into a permanent secret.

What to learn from this phase

- Foreign-key integrity.
- Authorization vs authentication.
- Replay attacks.
- Token expiry.
- Threat modeling a mobile app.

Checkpoint: Two real accounts pair successfully, while a third account cannot read or mutate their couple data.

Chapter 4 - Phase 3: Home + Avatar/Pet Presence

Goal: build the app's signature visual layer before wiring it to realtime events.

4.1 Onboarding

```
Account
-> Profile
-> Avatar
-> Pet
-> Pair
-> Home
```

4.2 Data-driven configuration

```
{
  "avatar": "default_full_body_01",
  "pet": "dog_01"
}
```

Store identifiers/configuration, not rendered images. V1 uses default full-body avatars and a small predefined pet list.

4.3 Home composition

```
Home
|
+-- PartnerPresence
|   +-- PartnerAvatar
|   +-- Pet
|   +-- AnimationState
|
+-- DailyStatus
|   +-- Outfit
|   +-- Photo
|   +-- Question
|
+-- ConnectionActions
    +-- LoveDrop
    +-- Mood
    +-- IWantToTalk
```

4.4 Animation state machine

```
IDLE
|
+--> WALKING
+--> PLAYING
+--> PETTING
+--> FEEDING
+--> SITTING
+--> RESTING
+--> SLEEPING
+--> REACTION
|
+-----> IDLE
```

Keep animation state in the presentation layer. Backend state such as 'partner selected a blue shirt' should enter the ViewModel, which converts it into presentation state.

4.5 Event-driven animation

```
enum AvatarEvent {
  loveReceived,
  hugReceived,
  goodMorning,
  goodNight,
  feedPet,
  petAnimal,
  playWithPet,
}
```

An event enters the state machine. The state machine decides the animation. Do not let ten widgets directly manipulate the same `AnimationController`.

4.6 First milestone

Make the partner avatar + pet feel alive without any network feature. If the app feels like a static profile page at this stage, fix that before proceeding.

Tech Stack Lesson: Flutter rendering + animations + state machines

What we are using

- Flutter widgets for composition.
- `AnimationController`/implicit animations as appropriate.
- `ViewModel` for presentation state.
- Finite-state-machine style event handling.
- Local bundled assets for V1.

Why this choice

The avatar is the emotional centerpiece, so it deserves a clean presentation boundary. Flutter's declarative model lets the UI reflect current state, while a state machine prevents competing animations from fighting. Keeping assets local makes the first version reliable and fast.

What to learn from this phase

- Widget rebuilds.
- Animation lifecycle.
- Transient UI state vs server state.
- Finite state machines.
- Asset lifecycle.

Checkpoint: The partner avatar and pet render correctly and transition among ambient animations without any backend dependency.

Chapter 5 · Phase 4: Realtime Connection Features

Goal: make the two phones feel connected through Love Drops, Mood Sync, and 'I Want to Talk'.

5.1 Realtime mental model

```
Postgres = durable truth
  |
  v
Supabase Realtime = change delivery
  |
  v
Repository Stream
  |
  v
ViewModel state
  |
  v
UI + avatar reaction
```

Think of realtime as a doorbell, not the house. The event says something changed; the database tells you what the current truth is.

5.2 Love Drop schema

```
love_drops
  id
  couple_id
  sender_id
  type
  message
  created_at
  read_at
```

5.3 Love Drop flow

- 1 User selects a quick affection type or custom text.
- 2 ViewModel validates input.
- 3 Repository inserts the record.
- 4 Postgres persists it.
- 5 Realtime informs the partner.
- 6 Partner repository updates its stream/state.
- 7 Partner UI renders it and can trigger a small avatar reaction.

The recipient does not have to respond. That is part of the product's low-pressure design.

5.4 Mood Sync

```
mood
  |
  +-- primary mood
  +-- optional need
  +-- optional context
  +-- updated_at
```

Example: 'Overwhelmed' + 'Just affection' + 'Work was horrible today.' The partner gets useful context without being assigned the job of fixing the mood.

5.5 I Want to Talk

```
connection_signal
  id
  sender_id
  receiver_id
```

```
type
  created_at
  expires_at
  acknowledged_at
```

Make this a transient signal. An expired signal should not look like an unresolved three-day-old crisis.

5.6 Reconnect

```
onReconnect:
  resubscribe()
  fetchCurrentState()
  mergeState()
  render()
```

Phones sleep and networks change. Realtime subscriptions will drop. Your repository must be able to recover from a fresh authoritative read.

Tech Stack Lesson: Supabase Realtime + Streams + event-driven UI

What we are using

- Postgres as persistent source of truth.
- Supabase Realtime for live delivery.
- Dart Streams for continuous state.
- Repositories as data boundaries.
- Private channels and couple authorization.

Why this choice

A custom websocket server is unnecessary for V1. Supabase Realtime gives the transport while PostgreSQL remains durable. This also teaches a core distributed-systems lesson: event delivery and authoritative state are different things.

What to learn from this phase

- Streams and subscriptions.
- WebSocket/change-feed concepts.
- Reconnect/resynchronization.
- Eventual consistency at the UI boundary.
- Idempotent event handling.

Checkpoint: Two physical devices can exchange Love Drops, moods, and talk signals in realtime and recover correctly after reconnect.

Chapter 6 - Phase 5: Daily Question Engine + Cron

Goal: implement the alternating custom question, hidden-answer rule, reveal condition, and idempotent daily job.

6.1 Data model

```
daily_connections
  id
  couple_id
  date
  creator_id
  question
  created_at

daily_answers
  id
  daily_connection_id
  user_id
  answer
  created_at
  unique(daily_connection_id, user_id)
```

6.2 Alternating creator

Store the initial creator on the couple and derive the turn from the number of elapsed couple-local days. This avoids a mutable 'whose turn' flag drifting out of sync.

```
days = current_local_date - couple_start_local_date

if days % 2 == 0:
    creator = initial_creator
else:
    creator = other_partner
```

Choose and document the timezone that defines 'day'. For V1, a shared couple timezone is reasonable if both users live in the same region.

6.3 Cron's job

```
Daily Cron
|
v
Find active couples
|
+--> determine local date
+--> determine creator
+--> create today's row if missing
|
v
Creator writes custom question
```

Cron should not invent a question. Its job is to establish daily state. The person whose turn it is supplies the content.

6.4 Idempotency

```
create unique index daily_one_per_couple_day
on daily_connections(couple_id, date);
```

If the job runs twice, the database prevents a duplicate. This is a simple but powerful distributed-systems technique: make repeated execution harmless.

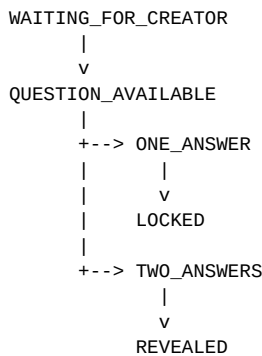
6.5 Answer reveal

Never return the partner's hidden answer to the client before both have answered. A UI lock is not security.

```
can_reveal(question_id):
    return answer_count(question_id) = 2
```

Implement this as a secure database function/RPC or equivalent server-side operation. The query should return only data the authenticated user is entitled to see.

6.6 State machine



6.7 Race cases

- Both partners submit at nearly the same time.
- One answer request retries.
- Cron runs twice.
- Creator never writes a question.
- Devices have different local dates.
- A user tries to query the hidden answer directly.
- Decide whether answers are editable before reveal and immutable afterward.

Tech Stack Lesson: PostgreSQL constraints + Cron + idempotency

What we are using

- Supabase Cron / pg_cron for recurring work.
- Postgres UNIQUE constraints.
- Database functions/RPC for secure business operations.
- Transactions and concurrency thinking.
- Timezone-aware daily state.

Why this choice

The daily question is a small feature with real backend complexity: duplicate jobs, simultaneous writes, hidden data, and day boundaries. Server-side rules are used because the client must never be trusted with the answer it is supposed to be unable to see.

What to learn from this phase

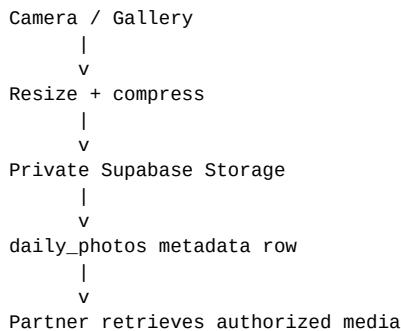
- Idempotency.
- Race conditions.
- Atomicity.
- Unique constraints as concurrency tools.
- Server-side authorization.

Checkpoint: The question turn alternates, hidden answers stay hidden until both answer, and repeated Cron execution cannot create duplicates.

Chapter 7 - Phase 6: Daily Photo + Outfit Colour Sync

Goal: implement the daily visual ritual and reflect the partner's outfit colours in their avatar.

7.1 Photo architecture



7.2 Never store photo bytes in Postgres

Store the image object in Storage and store metadata such as owner, date, and storage path in Postgres.

```
daily_photos
  id
  user_id
  date
  storage_path
  created_at
```

Use private storage and authorized/signed access. A database row should not become a public pointer to a private image.

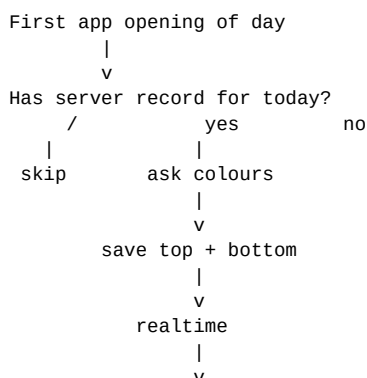
7.3 'At least one photo'

The product requires at least one photo per day, not exactly one. V1 can keep the interface focused on the first shared photo while the schema remains capable of multiple records.

7.4 Image handling

- Request camera/gallery permissions only when needed.
- Resize oversized images.
- Compress before upload.
- Retry failed uploads.
- Avoid duplicate metadata rows on retry.
- Handle permission denial with a clear explanation.

7.5 Outfit colours



The server record is authoritative. A local boolean is only an optimization because the user can reinstall the app or use another device.

7.6 Timezone consistency

Use the same couple/user timezone policy as the daily question. Otherwise one feature could call it Monday while another calls it Tuesday.

Tech Stack Lesson: Supabase Storage + private media + image processing

What we are using

- Supabase Storage for photo objects.
- Postgres for photo metadata and daily state.
- Signed/authorized access to private media.
- Flutter image_picker for camera/gallery.
- Client-side compression before upload.

Why this choice

Binary media and relational state have different storage needs. Object storage handles photos while PostgreSQL handles ownership/date/metadata. The separation is a classic backend boundary and keeps the database focused on structured data.

What to learn from this phase

- Object storage vs relational storage.
- Signed URLs/private access.
- Upload retries.
- Image compression.
- Server-authoritative daily state.

Checkpoint: Each partner can securely share a daily photo and the selected outfit colours reliably update the partner avatar.

Chapter 8 · Phase 7: Notifications + UX Polish

Goal: make the app useful when closed without turning notifications into pressure.

8.1 Notification matrix

Event	Example	Policy
Love Drop	♥■ You received a Love Drop	Immediate
Daily question	■ Today's question is ready	Daily
Partner answered	Your partner answered	Never expose answer
Reveal	■ Both answers are ready	Immediate
Daily photo	■ Share a little piece of your day	Gentle reminder
Mood/Talk	■ Your partner needs affection	Timely, not noisy

8.2 Realtime vs push

Realtime is for active synchronization while the app is running. Push notifications are for attention when the app is backgrounded or closed. Do not force a websocket to act like a push-notification service.

8.3 Notification privacy

Do not put private custom Love Drop text on the lock screen by default. A safe notification can say that a Love Drop arrived, leaving the private content inside the app.

8.4 UX states

- Loading.
- Success.
- Empty.
- Retryable error.
- Offline.
- Permission denied.
- Unauthorized/session expired.
- Background/resume reconciliation.

8.5 Cross-platform checks

- Notification permission flows.
- Photo-picker behavior.
- Safe areas and keyboard.
- Background/resume.
- Animation performance.
- Accessibility and text scaling.
- Reduced-motion behavior where practical.

Tech Stack Lesson: Push notifications + mobile lifecycle

What we are using

- Realtime for foreground synchronization.
- Push notifications for background delivery.
- Platform notification permissions.
- Flutter lifecycle handling.
- Resilient loading/error/empty states.

Why this choice

Mobile applications are frequently suspended. Realtime cannot guarantee that the partner's app is actively listening. Push solves the background-attention problem. Keeping notification text conservative also protects private relationship content on lock screens.

What to learn from this phase

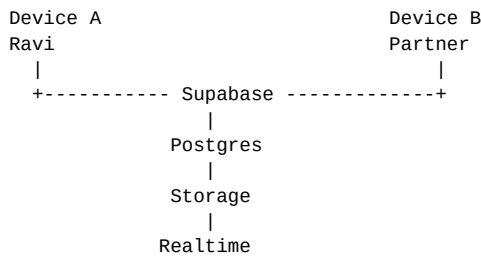
- Foreground vs background execution.
- Push vs realtime.
- Notification payload privacy.
- Permission models.
- Lifecycle-driven state refresh.

Checkpoint: Important events reach the partner when appropriate without excessive or privacy-leaking notifications.

Chapter 9 · Phase 8: End-to-End Testing + Release

Goal: prove the product works as a private two-device system, not merely as a collection of screens.

9.1 Test with two real accounts



Use two actual accounts. Mocking the partner is useful for unit tests but will miss authorization, ordering, reconnect, and concurrency bugs.

9.2 Authentication

Scenario	Expected
Valid signup/login	Session established
Duplicate email	Clear failure
Restart app	Session restored
Sign out	Protected screens inaccessible
Network failure	Graceful retry/error

9.3 Security

- A and B can access only their couple.
- C cannot read the couple.
- C cannot insert Love Drops into the couple.
- C cannot read daily photos.
- C cannot read hidden answers.
- Pairing token cannot be replayed.
- Expired pairing token cannot be used.

9.4 Daily question

Scenario	Expected
A's turn	A can create
B's turn	B can create
Only A answered	B's answer hidden
Only B answered	A's answer hidden
Both answered	Both visible
Duplicate answer	No duplicate row

Scenario	Expected
Cron twice	One daily record
Reconnect	Correct current state

9.5 Photo + avatar

- Camera permission allowed/denied.
- Gallery selection works.
- Large image compressed.
- Upload interruption retries.
- Unauthorized media access fails.
- Outfit colours persist across app restart.
- Partner avatar changes after outfit update.
- Ambient animations do not block interaction.

9.6 Observability

Log structured errors for auth, database, storage, notification, and Cron failures. Never log passwords, access tokens, private message bodies, or private media URLs.

9.7 Release sequence

- 1 Development Supabase + debug builds.
- 2 Internal Android build.
- 3 Internal iOS/TestFlight build where available.
- 4 Two-device end-to-end pass.
- 5 RLS/security review.
- 6 Performance and battery pass.
- 7 Production Supabase project.
- 8 Production environment configuration.
- 9 Private release to intended users.
- 10 Observe and fix before any broader distribution.

Tech Stack Lesson: Testing, environments, security, operations

What we are using

- Unit tests for ViewModels and business rules.
- Repository/service tests.
- RLS/security tests.
- Two-device integration/end-to-end tests.
- Separate development and production Supabase projects.
- Privacy-safe structured logs.

Why this choice

Distributed apps often fail between components rather than inside a single function. Testing both devices and the authorization boundary catches the bugs that matter most. Separate environments prevent test data from becoming production data.

What to learn from this phase

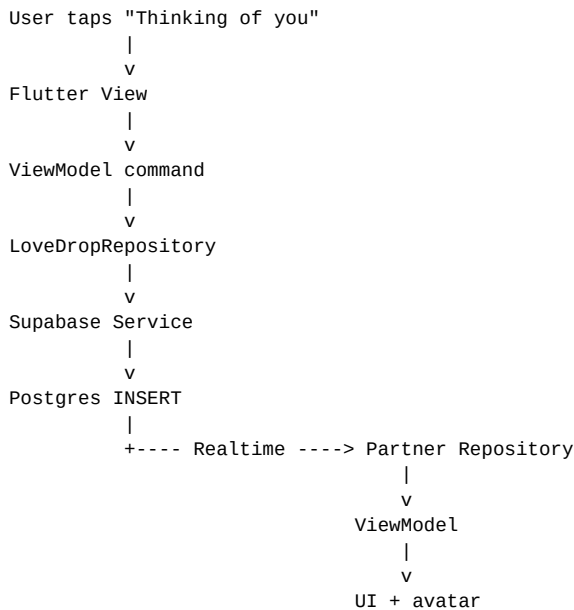
- Test pyramid.
- Integration vs end-to-end tests.
- Security testing.
- Environment separation.
- Operational logging.

Checkpoint: The full V1 loop works on two real devices, unauthorized access fails, and production is isolated from development.

Chapter 10 - Architecture Deep Dive

This chapter explains the design decisions you should be able to defend in an interview or code review.

10.1 End-to-end request



10.2 Why repositories?

A repository is the feature's source-of-truth boundary. It hides whether data comes from Supabase, cache, or another service. That makes the ViewModel testable and prevents a widget from knowing how the database works.

10.3 Where business rules belong

Rule	Location
Show loading spinner	View/ViewModel
Format date for UI	ViewModel/presentation helper
User belongs to couple	Database/RLS
Both answers required to reveal	Database/server operation
One question per couple/day	Database unique constraint
Question creator alternation	Server-side deterministic rule
Image compression	Client
Private image access	Storage + authorization
Avatar animation	Presentation state machine

10.4 Why not Spring Boot for V1?

Spring Boot would be a valid choice and matches your backend background, but it would make V1 responsible for more infrastructure: API deployment, authentication integration, websocket/realtime infrastructure, storage integration, and scheduled workers. Supabase gives those primitives while retaining PostgreSQL underneath. This is a scope and learning choice, not a statement that Spring Boot is worse.

10.5 Why Flutter rather than native Android + iOS?

Native clients maximize platform-specific control but duplicate much of the implementation. This V1 is small, private, and animation-heavy. Flutter reduces duplication while still allowing platform integration when necessary.

10.6 Why not build chat?

The product assumes normal calls, texts, and video calls continue outside the app. V1's differentiation is ambient presence, daily rituals, affection signals, and shared micro-moments. A full chat system would increase scope without improving that central idea.

10.7 The storage boundary

Auth	-> identity/session
Postgres	-> structured persistent state
Storage	-> photo binaries
Realtime	-> change delivery
Cron	-> recurring backend work
Flutter	-> presentation + transient state

If you can explain why each piece owns exactly one responsibility, you understand the architecture. If everything can call everything, you don't.

Chapter 11 · Learning Curriculum

Use the project as a hands-on course. Learn each concept immediately before the phase that needs it.

Phase	Learn	Ready when...
0	Dart async, Flutter widgets, Git	You can explain a widget tree and Future/Stream.
1	Auth, PostgreSQL, RLS	You can explain why client-side security is insufficient.
2	FKs, constraints, tokens	You can secure couple membership.
3	State + animations + FSM	You can separate animation state from server state.
4	Streams, realtime, reconnect	You can explain event vs source of truth.
5	Cron, idempotency, transactions, timezones	You can defend the daily-question design.
6	Object storage, signed access, compression	You can explain media architecture.
7	Push, lifecycle, permissions	You can distinguish push from realtime.
8	Testing, RLS tests, release envs	You can prove two-device correctness.

The learning method

- 1 Learn the minimum concept required to implement the next feature.
- 2 Implement a small slice.
- 3 Break it intentionally and observe the failure.
- 4 Read the underlying technology documentation.
- 5 Refactor once the mental model is clear.
- 6 Write a short note explaining the design in your own words.

Do not over-learn before building

You do not need to master all of Flutter, Supabase, PostgreSQL, or mobile development before starting. Learn the smallest useful piece, build it, then deepen the model when a real engineering problem appears. This project is your laboratory.

Chapter 12 · Master V1 Checklist

A feature is checked only after it is tested.

Phase 0

- ■ Flutter Android build
- ■ Flutter iOS build where available
- ■ Git
- ■ Architecture folders
- ■ Riverpod
- ■ Routing
- ■ Configuration

Phase 1

- ■ Supabase project
- ■ Auth
- ■ Session restore
- ■ users table
- ■ RLS
- ■ Unauthorized profile test

Phase 2

- ■ Couple table
- ■ Create/Join
- ■ Token expiry
- ■ Replay prevention
- ■ Couple RLS
- ■ Third-user denial

Phase 3

- ■ Avatar setup
- ■ Pet selection
- ■ Partner rendering
- ■ Ambient animations
- ■ Animation state machine

Phase 4

- ■ Quick Love Drop
- ■ Custom Love Drop

- ■ Realtime
- ■ Mood Sync
- ■ I Want to Talk
- ■ Reconnect/resync

Phase 5

- ■ Daily question
- ■ Alternating creator
- ■ Custom question
- ■ Hidden answers
- ■ Reveal
- ■ Cron
- ■ Idempotency
- ■ Timezone

Phase 6

- ■ Private photo storage
- ■ Camera/gallery
- ■ Compression
- ■ Photo metadata
- ■ Partner access
- ■ Daily outfit
- ■ Avatar colour update

Phase 7

- ■ Push setup
- ■ Love Drop notification
- ■ Daily question notification
- ■ Answer privacy
- ■ Reveal notification
- ■ Gentle photo reminder
- ■ Notification settings

Phase 8

- ■ Two-device E2E
- ■ RLS attack tests
- ■ Offline/reconnect
- ■ Media permission tests

- ■ Performance
- ■ Privacy-safe logs
- ■ Production separation

Appendix · References & Source Basis

The V1 requirements are derived from the locked PRD; technology guidance is expanded using current official documentation.

Primary source: Two-Person Connection App — V1 Product Requirements Document. This guide preserves its V1 scope, daily question rules, daily photo requirement, mood system, avatar/pet concept, realtime requirements, data model direction, security expectations, and initial technology choice.

Flutter official documentation consulted: App Architecture; Guide to App Architecture; Architectural Overview; State Management; Learn Flutter. These support the use of separation of concerns, Views/ViewModels, Repositories/Services, declarative UI, and scalable Flutter architecture.

Supabase official documentation consulted: Use Supabase with Flutter; Flutter installation/client documentation; User Management with Flutter; Realtime Getting Started. These support Supabase initialization, PostgreSQL/RLS patterns, Storage, and Realtime concepts.

Version note: package versions, platform requirements, free-tier quotas, and dashboard labels can change. Verify current official documentation before production deployment.

Official documentation

- Flutter: docs.flutter.dev
- Supabase: supabase.com/docs
- Supabase Flutter quickstart and Realtime documentation.